



## Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

### STUDENT CASE STUDY EXECUTION REPORT

|                |                                                    |
|----------------|----------------------------------------------------|
| Department     | FED                                                |
| Subject        | DSA-2                                              |
| Academic Year  | I-III Semester(2025-2026)                          |
| Faculty Name   | Ms. G. Mamatha                                     |
| Student Name   | K. Sai Samyuktha                                   |
| Roll Number    | 2520030443                                         |
| Branch         | CSE                                                |
| Course Outcome | CO-1 Binary Search Tree, AVL Tree , Tree Traversal |

**Case Study Topic:** Efficient Patient Record Management using Binary Search Tree (BST) and AVL Tree in HealthNet System

#### Work Done Summary:

In this case study, patient records in the HealthNet system are managed using Binary Search Tree (BST) and AVL Tree data structures. BST is used for basic insertion, searching, and deletion of patient data, while AVL Tree ensures the database remains balanced for efficient operations. Tree traversal techniques are used to display patient records in sorted order and improve data accessibility.

#### Implementation Details:

1. Patient records are stored using Binary Search Tree (BST) with Patient ID as the key.
2. Each node contains patient information such as ID, name, age, disease, and treatment history.
3. Insertion operation adds new patient records into the BST based on Patient ID comparison.
4. Search operation retrieves patient details in  $O(\log n)$  average time (balanced case).
5. Deletion operation removes patient records when treatment is completed or patient is discharged.
6. To avoid imbalance in the BST, an AVL Tree is used, which performs rotations (LL, RR, LR, RL) to maintain height balance.

7. Inorder traversal is used to display patient records in sorted order of Patient ID.
8. The system ensures efficient performance even when large volumes of patient data are added or removed frequently.

### Result : Screenshots / Output

```
? Final AVL Tree Structure:

└─ 30 (Ravi)
   └─ 40 (Meena)
      └─ 50 (John)
   └─ 20 (Kiran)
      └─ 25 (Sara)
      └─ 10 (Anita)

? Inorder Traversal (Sorted Order):

10 (Anita) 20 (Kiran) 25 (Sara) 30 (Ravi) 40 (Meena) 50 (John)
```

1. Insertion:  $O(\log n)$  in AVL tree due to balancing,  $O(n)$  in worst-case BST
2. Search:  $O(\log n)$  in AVL tree because of height-balanced structure
3. Deletion:  $O(\log n)$  in AVL tree including rotation operations
4. Inorder Traversal:  $O(n)$  as all nodes are visited once
5. AVL Tree keeps the system balanced and ensures fast patient record operations with  $O(\log n)$  efficiency.

**GitHub Link:** <https://github.com/saisamyuktha471/DSA2-CO-1>

| Student Signature | Faculty Signature |
|-------------------|-------------------|
|                   |                   |